

LAPORAN PRATIKUM
STRUKTUR DATA
“SINGLE LINKED LIST”

PEKAN KE-5

disusun Oleh:

Aisyah Najmiatul Fauziah

NIM 2511533030

Dosen Pengampu: DR. WAHYUDI, S.T, M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum Struktur Data dengan topik *Single Linked List* ini dengan baik.

Laporan ini disusun sebagai salah satu bentuk tugas praktikum yang bertujuan untuk memahami konsep dasar struktur data, khususnya dalam implementasi *Single Linked List*. Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun pemahaman materi.

Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menambah wawasan dalam bidang struktur data.

Padang, 7 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Praktikum “NodeSLL_2511533030”.....	2
2.2 Praktikum “PencarianSLL_2511533030”	2
2.3 Praktikum “TambahSLL_2511533030”.....	2
2.4 Pratikum “HapusSLL_2511533030.....	3
BAB III.....	3
KESIMPULAN.....	3
3.1 Kesimpulan.....	3
DAFTAR PUSTAKA.....	4

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan menyimpan data agar dapat diakses dan diolah secara efisien. Salah satu struktur data yang sering digunakan adalah *Linked List*, khususnya *Single Linked List*. *Single Linked List* adalah struktur data linear yang terdiri dari kumpulan node, di mana setiap node memiliki data dan satu referensi (pointer) ke node berikutnya. Berbeda dengan array, struktur ini lebih fleksibel dalam hal penambahan dan penghapusan data karena tidak memerlukan alokasi memori yang berurutan.

Praktikum ini dilakukan untuk memberikan pemahaman secara langsung mengenai cara kerja *Single Linked List*, mulai dari pembuatan node, penyisipan data, penghapusan data, hingga penelusuran elemen.

1.2 Tujuan Pratikum

1. Memahami konsep dasar *Single Linked List*.
2. Mempelajari cara membuat dan mengimplementasikan *Single Linked List* dalam program.
3. Mengetahui cara menambahkan, menghapus, dan menampilkan data dalam *Linked List*.

1.3 Manfaat Pratikum

1. Mahasiswa dapat memahami konsep struktur data secara lebih mendalam, khususnya *Single Linked List*.
2. Meningkatkan kemampuan dalam pemrograman, terutama dalam pengolahan data dinamis.
3. Membantu dalam menyelesaikan permasalahan yang membutuhkan struktur data fleksibel.

BAB II PEMBAHASAN

2.1 Pratikum “NodeSLL_2511533030”

Langkah Kerja :

1. Buat projek, Package, dan Class baru
2. Buat kode berikut

```
package pekan5_2511533030;

public class NodeSLL_2511533030 {
    // node bagian data
    int data_3030;
    // Pointer ke node berikutnya
    NodeSLL_2511533030 next_3030;
    // Konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533030(int data_3030)
    {
        this.data_3030 = data_3030;
        this.next_3030 = null;
    }
}
```

Gambar 2.1 kode program “NodeSLL_2511533030”

Penjelasan Kode :

2.1.1 Package

- Package berfungsi untuk mengelompokkan file program agar lebih terstruktur dan mudah dikelola.

```
package pekan5_2511533030;
```

2.1.2 Deklarasi Class Node Single Linked List

- Kata public berarti class dapat digunakan atau diakses oleh class lain.

```
public class NodeSLL_2511533030 {
```

2.1.3 Variabel Data pada Node

- Variabel data_3030 digunakan untuk menyimpan data pada node.
- Tipe data yang digunakan adalah int, sehingga data yang disimpan berupa bilangan bulat.

```
// node bagian data
int data_3030;
```

2.1.4 Pointer ke Node Berikutnya

- Variabel next_3030 digunakan sebagai penghubung atau penunjuk ke node berikutnya dalam linked list.
- Jika node belum terhubung ke node lain, maka nilainya null.

```
// Pointer ke node berikutnya
NodeSLL_2511533030 next_3030;
```

2.1.5 Konstruktor Node

- Konstruktor adalah method khusus yang otomatis dijalankan saat object dibuat.
- Konstruktor ini menerima parameter data_3030 untuk mengisi nilai data pada node.

```
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511533030(int data_3030)
```

2.1.6 Inisialisasi Data Node

- `this.data_3030` → variabel milik object/class
- `data_3030` → parameter dari konstruktor

```
{  
    this.data_3030 = data_3030;  
}
```

2.1.7 Inisialisasi Pointer Node

- Baris ini mengatur agar pointer `next_3030` bernilai null saat node pertama kali dibuat
- Artinya node belum terhubung ke node lain.

```
this.next_3030 = null
```

2.2 Pratikum “PencarianSLL_2511533030”

Langkah Kerja :

1. Buat Class baru “PencarianSLL_2511533030”
2. Buat kode program berikut

```
package pekan5_2511533030;  
  
public class PencarianSLL_2511533030 {  
    static boolean searchKey_3014 (NodeSLL_2511533030 head, int key) {  
        NodeSLL_2511533030 curr_3030 = head;  
        while (curr_3030 != null) {  
            if (curr_3030.data_3030 == key)  
                return true;  
            curr_3030 = curr_3030.next_3030; }  
        return false; }  
    public static void traversal_3014 (NodeSLL_2511533030 head) {  
        //mulai dari head  
        NodeSLL_2511533030 curr_3030 = head;  
        // telusuri sampai pointer null  
        while (curr_3030 != null) {  
            System.out.print(" " + curr_3030.data_3030);  
            curr_3030 = curr_3030.next_3030; }  
        System.out.println();  
    }  
    public static void main (String[] args) {  
        NodeSLL_2511533030 head = new NodeSLL_2511533030(14);  
        head.next_3030 = new NodeSLL_2511533030(21);  
        head.next_3030.next_3030 = new NodeSLL_2511533030(13);  
        head.next_3030.next_3030.next_3030 = new  
NodeSLL_2511533030(30);  
        head.next_3030.next_3030.next_3030.next_3030 = new  
NodeSLL_2511533030(10);  
        System.out.print ("Penelusuran SLL : ") ;  
        traversal_3014 (head);  
        // data yang akan dicari  
        int key = 30;  
        System.out.print("cari data " + key+ " = ");  
        if (searchKey_3014(head, key))  
            System.out.println("ketemu");  
        else  
            System.out.println("tidak ada");  
    }  
}
```

```
}  
}
```

Gambar 2.2 kode program “PencarianSLL_2511533030”

3. Didapatkan output sebagai berikut

```
<terminated> PencarianSLL_2511533030 [Java A  
Penelusuran SLL : 14 21 13 30 10  
cari data 30 = ketemu
```

Gambar 2.2 output program “PencarianSLL_2511533030”

Penjelasan kode :

2.2.1 Package

- Digunakan untuk menunjukkan bahwa program berada dalam package pekan5_2511533030.

```
package pekan5_2511533030;
```

2.2.2 Deklarasi Class

- Class ini digunakan untuk membuat program pencarian data pada struktur data *Single Linked List*.

```
public class PencarianSLL_2511533030 {
```

2.2.3 Method Search Data

- Method ini digunakan untuk mencari data tertentu pada linked list.
- Program akan menelusuri node satu per satu hingga data ditemukan atau linked list habis.

```
static boolean searchKey_3014 (NodeSLL_2511533030 head, int key) {  
    NodeSLL_2511533030 curr_3030 = head;  
    while (curr_3030 != null) {  
        if (curr_3030.data_3030 == key)  
            return true;  
        curr_3030 = curr_3030.next_3030; }  
    return false; }
```

2.2.4 Method Traversal

- Method traversal digunakan untuk menampilkan seluruh data yang ada pada linked list dari awal sampai akhir.

```
public static void traversal_3014 (NodeSLL_2511533030 head) {  
    //mulai dari head  
    NodeSLL_2511533030 curr_3030 = head;  
    // telusuri sampai pointer null  
    while (curr_3030 != null) {  
        System.out.print(" " + curr_3030.data_3030);  
        curr_3030 = curr_3030.next_3030; }  
    System.out.println(); }  
}
```

2.2.5 Method Main

- Method utama yang digunakan untuk menjalankan program.

```
public static void main (String[] args) {
```

2.2.6 Pembuatan Linked List

- Kode ini digunakan untuk membuat node-node dan menghubungkannya menjadi sebuah *Single Linked List*.

```
NodeSLL_2511533030 head = new NodeSLL_2511533030(14);
    head.next_3030 = new NodeSLL_2511533030(21);
    head.next_3030.next_3030 = new NodeSLL_2511533030(13);
    head.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(30);
    head.next_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(10);
```

2.2.7 Menampilkan Linked List

- Digunakan untuk menampilkan seluruh isi linked list menggunakan method traversal.

```
System.out.print ("Penelusuran SLL : " );
    traversal_3014 (head);
```

2.2.8 Proses Pencarian data

- Kode ini digunakan untuk mencari data 30 pada linked list dan menampilkan hasil pencariannya apakah ditemukan atau tidak.

```
// data yang akan dicari
    int key = 30;
    System.out.print("cari data " + key+ " = ");
    if (searchKey_3014(head, key))
        System.out.println("ketemu");
    else
        System.out.println("tidak ada");
```

2.3 Pratikum “TambahSLL_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat kode program berikut

```
package pekan5_2511533030;

public class TambahSLL_2511533030 {
    public static NodeSLL_2511533030 insertAtFront_3030
(NodeSLL_2511533030 head_3030, int value_3030) {
        NodeSLL_2511533030 new_node = new NodeSLL_2511533030
(value_3030);
        new_node.next_3030 = head_3030;
        return new_node;
    }
    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533030 insertAtEnd_3030
(NodeSLL_2511533030 head_3030, int value_3030) {
        // buat sebuah node dengan sebuah nilai
        NodeSLL_2511533030 newNode = new NodeSLL_2511533030
(value_3030);
        // jika list kosong maka node jadi head
        if (head_3030 == null) {
            return newNode;
        }
        // Simpan head ke variabel sementara
        NodeSLL_2511533030 last = head_3030;
```



```

        // telusuri ke node akhir
        while (last.next_3030 != null) {
            last = last.next_3030;
        }
        // ubah pointer
        last.next_3030 = newNode;
        return head_3030;
    }

    static NodeSLL_2511533030 GetNode (int data_3014) {
        return new NodeSLL_2511533030 (data_3014);
    }

    static NodeSLL_2511533030 insertPos (NodeSLL_2511533030 headNode,
int position_3030, int value_3030) {
        NodeSLL_2511533030 head_3030 = headNode;
        if (position_3030 < 1)
            System.out.print ("Invalid position");
        if (position_3030 == 1) {
            NodeSLL_2511533030 new_node = new
NodeSLL_2511533030(value_3030);
            new_node.next_3030 = head_3030;
            return new_node;
        } else {
            while (position_3030-- != 0) {
                if (position_3030 == 1) {
                    NodeSLL_2511533030 newNode = GetNode
(value_3030);

                    newNode.next_3030 = headNode.next_3030;
                    headNode.next_3030 = newNode;
                    break;
                }
                headNode = headNode.next_3030;
            }
            if (position_3030 != 1)
                System.out.print("Posisi di luar jangkauan");
            return head_3030; } }

    public static void printList_3014 (NodeSLL_2511533030 head_3030) {
        NodeSLL_2511533030 curr_3030 = head_3030;
        while (curr_3030.next_3030 != null) {
            System.out.print(curr_3030.data_3030+"-->");
            curr_3030 = curr_3030.next_3030;
        }
        if (curr_3030.next_3030==null) {
            System.out.print(curr_3030.data_3030); }
        System.out.println();
    }

    public static void main (String[] args) {
        // buat linked list 2->3->5->6
        NodeSLL_2511533030 head_3030 = new NodeSLL_2511533030(2);
        head_3030.next_3030 = new NodeSLL_2511533030(3);
        head_3030.next_3030.next_3030 = new NodeSLL_2511533030(5);
        head_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(6);
        // cetak list asli
        System.out.print("Senarai berantai awal:");
        printList_3014 (head_3030);
        // tambahkan node baru didepan
        System.out.print("tambah 1 simpul di depan: ");
    }

```

```

        int data_3030 = 1;
        head_3030 = insertAtFront_3030(head_3030, data_3030);
        // cetak update list
        printList_3014(head_3030);
        // tambahkan node baru dibelakang
        System.out.print("tambah 1 simpul di belakang");
        int data2 = 7;
        head_3030 = insertAtEnd_3030 (head_3030, data2);
        // cetak update list
        printList_3014 (head_3030);
        System.out.print("tambah 1 simpul ke data 4: ");
        int data3 = 4;
        int pos=4;
        head_3030 = insertPos (head_3030,pos,data3);
        // cetak update list
        printList_3014 (head_3030);
    }
}

```

Gambar 2.3 kode program “TambahSLL_2511533030”

3. Didapatkan output sebagai berikut

```

<terminated> TambahSLL_2511533030 [Java Application] C:\Users\AISYAH
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

Gambar 2.3 kode program “TambahSLL_2511533030”

Penjelasan Kode :

2.3.1 Package

- Digunakan untuk menunjukkan bahwa program berada dalam package pekan5_2511533030.

```
package pekan5_2511533030;
```

2.3.2 Deklarasi Class

- Class ini digunakan untuk membuat program penambahan node pada struktur data *Single Linked List*.

```
public class TambahSLL_2511533030 {
```

2.3.3 Method Insert At Front

- Method ini digunakan untuk menambahkan node baru di bagian depan linked list sehingga node baru menjadi head.

```

// fungsi menambahkan node di akhir SLL
public static NodeSLL_2511533030 insertAtEnd_3030
(NodeSLL_2511533030 head_3030, int value_3030) {
    // buat sebuah node dengan sebuah nilai
    NodeSLL_2511533030 newNode = new NodeSLL_2511533030
(value_3030);

    // jika list kosong maka node jadi head
    if (head_3030 == null) {
        return newNode;
    }
}

```

```
}
```

2.3.4 Method Insert At End

- Method ini digunakan untuk menambahkan node baru di bagian akhir linked list.

```
// Simpan head ke variabel sementara
NodeSLL_2511533030 last = head_3030;
// telusuri ke node akhir
while (last.next_3030 != null) {
    last = last.next_3030;
}
// ubah pointer
last.next_3030 = newNode;
return head_3030;
}
```

2.3.5 Method GetNode

- Method ini digunakan untuk membuat node baru dengan nilai tertentu.

```
static NodeSLL_2511533030 GetNode (int data_3014) {
    return new NodeSLL_2511533030 (data_3014);
}
```

2.3.6 Method Insert Position

- Method ini digunakan untuk menambahkan node pada posisi tertentu di linked list.

```
static NodeSLL_2511533030 insertPos (NodeSLL_2511533030 headNode, int
position_3030, int value_3030) {
```

2.3.7 Method Print List

- Method ini digunakan untuk menampilkan seluruh isi linked list.

```
public static void printList_3014 (NodeSLL_2511533030
head_3030) {
```

2.3.8 Method Main

- Method utama yang digunakan untuk menjalankan program.

```
public static void main (String[] args) {
```

2.3.9 Pembuatan Linked List

- Kode ini digunakan untuk membuat node-node dan menghubungkannya menjadi sebuah *Single Linked List*.

```
// buat linked list 2->3->5->6
NodeSLL_2511533030 head_3030 = new NodeSLL_2511533030(2);
head_3030.next_3030 = new NodeSLL_2511533030(3);
head_3030.next_3030.next_3030 = new NodeSLL_2511533030(5);
head_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(6);
```

2.3.10 Menampilkan Linked List Awal

- Digunakan untuk menampilkan isi linked list awal sebelum dilakukan penambahan node.

```
// cetak list asli
System.out.print("Senarai berantai awal:");
printList_3014 (head_3030);
```

2.3.11 Penambahan Node di depan

- Kode ini digunakan untuk menambahkan node 1 di bagian depan linked list lalu menampilkan hasilnya.

```
// tambahkan node baru didepan
System.out.print("tambah 1 simpul di depan: ");
int data_3030 = 1;
head_3030 = insertAtFront_3030(head_3030, data_3030);
```

2.3.12 Penambahan Node di belakang

- Kode ini digunakan untuk menambahkan node 7 di bagian akhir linked list lalu menampilkan hasilnya.

```
// tambahkan node baru dibelakang
System.out.print("tambah 1 simpul di belakang");
int data2 = 7;
head_3030 = insertAtEnd_3030 (head_3030, data2);
```

2.3.13 Penambahan Node di posisi tertentu

- Kode ini digunakan untuk menambahkan node 4 pada posisi ke-4 linked list lalu menampilkan hasilnya.

```
System.out.print("tambah 1 simpul ke data 4: ");
int data3 = 4;
int pos=4;
head_3030 = insertPos (head_3030,pos,data3)
```

2.4 Pratikum “HapusSLL_2511533030”

Langkah Kerja :

- Buat Class baru
- Buat kode program berikut

```
package pekan5_2511533030;

public class HapusSLL_2511533030 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511533030 deleteHead_3014(NodeSLL_2511533030 head_3030)
    {
        // jika SLL kosong
        if (head_3030 == null) {
            return null;
        }
        // pindahkan head ke node berikutnya
        head_3030 = head_3030.next_3030;
        // return head baru
        return head_3030;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533030 removeLastNode_3014(NodeSLL_2511533030
head_3030) {
        // jika list kosong , return null
        if (head_3030 == null) {
            return null;
        }
    }
}
```

```

    }
    // jika list satu node , hapus node dan return null
    if (head_3030.next_3030 == null) {
        return null;
    }
    // temukan node terakhir kedua
    NodeSLL_2511533030 secondLast = head_3030;
    while (secondLast.next_3030.next_3030 != null) {
        secondLast = secondLast.next_3030;
    }
    // hapus node terakhir
    secondLast.next_3030 = null;
    return head_3030;
}

// fungsi menghapus node di posisi tertentu
public static NodeSLL_2511533030 deleteNode_3014(NodeSLL_2511533030 head_3030,
int position_3030) {
    NodeSLL_2511533030 temp_3030 = head_3030;
    NodeSLL_2511533030 prev_3030 = null;

    // jika linked list null
    if (temp_3030 == null)
        return head_3030;

    // kasus 1: head dihapus
    if (position_3030 == 1) {
        head_3030 = temp_3030.next_3030;
        return head_3030;
    }

    // kasus 2: menghapus node di tengah
    for (int i_3030 = 1; temp_3030 != null && i_3030 < position_3030;
i_3030++) {
        prev_3030 = temp_3030;
        temp_3030 = temp_3030.next_3030;
    }

    // jika ditemukan, hapus node
    if (temp_3030 != null) {
        prev_3030.next_3030 = temp_3030.next_3030;
    } else {
        System.out.println("Data tidak ada");
    }
    return head_3030;
}

// fungsi mencetak SLL
public static void printList_3030(NodeSLL_2511533030 head_3030) {
    NodeSLL_2511533030 curr_3030 = head_3030;
    while (curr_3030.next_3030 != null) {
        System.out.print(curr_3030.data_3030 + "-->");
        curr_3030 = curr_3030.next_3030;
    }
    System.out.print(curr_3030.data_3030);
    System.out.println();
}

// kelas main

```

```

public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511533030 head_3030 = new NodeSLL_2511533030(1);
    head_3030.next_3030 = new NodeSLL_2511533030(2);
    head_3030.next_3030.next_3030 = new NodeSLL_2511533030(3);
    head_3030.next_3030.next_3030.next_3030 = new NodeSLL_2511533030(4);
    head_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(5);
    head_3030.next_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(6);

    // cetak list awal
    System.out.println("List awal: ");
    printList_3030(head_3030);

    // hapus head
    head_3030 = deleteHead_3014(head_3030);
    System.out.println("List setelah head dihapus: ");
    printList_3030(head_3030);

    // hapus node terakhir
    head_3030 = removeLastNode_3014(head_3030);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList_3030(head_3030);

    // hapus node di posisi 2
    int position_3030 = 2;
    head_3030 = deleteNode_3014(head_3030, position_3030);
    System.out.println("List setelah posisi 2 dihapus: ");
    printList_3030(head_3030);
}
}

```

Gambar 2.4 kode program “HapusSLL_2511533030”

3. Didapatkan output program sebagai berikut

```

<terminated> HapusSLL_2511533030 [Java Application] C:\Users\AISYAH NAJMIATUL
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

Gambar 2.4 output program “HapusSLL_2511533030”

Penjelasan Kode :

2.4.1 Package

- Digunakan untuk menunjukkan bahwa program berada dalam package pekan5_2511533030.

```
package pekan5_2511533030;
```

2.4.2 Deklarasi Class

- Class ini digunakan untuk membuat program penghapusan node pada struktur data *Single Linked List*.

```
public class HapusSLL_2511533030 {
```

2.4.3 Method Delete Head

- Method ini digunakan untuk menghapus node paling depan (head) pada linked list.

```
// fungsi untuk menghapus head
public static NodeSLL_2511533030 deleteHead_3014(NodeSLL_2511533030 head_3030)
{
    // jika SLL kosong
    if (head_3030 == null) {
        return null;
    }
    // pindahkan head ke node berikutnya
    head_3030 = head_3030.next_3030;
    // return head baru
    return head_3030;
}
```

2.4.4 Method Remove Last Node

- Method ini digunakan untuk menghapus node paling akhir pada linked list.

```
// fungsi menghapus node terakhir SLL
public static NodeSLL_2511533030 removeLastNode_3014(NodeSLL_2511533030
head_3030) {
    // jika list kosong , return null
    if (head_3030 == null) {
        return null;
    }
    // jika list satu node , hapus node dan return null
    if (head_3030.next_3030 == null) {
        return null;
    }
    // temukan node terakhir kedua
    NodeSLL_2511533030 secondLast = head_3030;
    while (secondLast.next_3030.next_3030 != null) {
        secondLast = secondLast.next_3030;
    }
    // hapus node terakhir
    secondLast.next_3030 = null;
    return head_3030;
}
```

2.4.5 Method Delete Node Position

- Method ini digunakan untuk menghapus node pada posisi tertentu di linked list.

```
// fungsi menghapus node di posisi tertentu
public static NodeSLL_2511533030 deleteNode_3014(NodeSLL_2511533030 head_3030,
int position_3030) {
```

2.4.6 Method Print List

- Method ini digunakan untuk menampilkan seluruh isi linked list.

```
public static void printList_3030(NodeSLL_2511533030 head_3030) {
```

2.4.7 Method Main

- Method utama yang digunakan untuk menjalankan program.

```
public static void main(String[] args) {
```

2.4.8 Pembuatan Linked List

- Kode ini digunakan untuk membuat node-node dan menghubungkannya menjadi sebuah *Single Linked List*.

```
// buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
NodeSLL_2511533030 head_3030 = new NodeSLL_2511533030(1);
head_3030.next_3030 = new NodeSLL_2511533030(2);
head_3030.next_3030.next_3030 = new NodeSLL_2511533030(3);
head_3030.next_3030.next_3030.next_3030 = new NodeSLL_2511533030(4);
head_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(5);
head_3030.next_3030.next_3030.next_3030.next_3030.next_3030 = new
NodeSLL_2511533030(6);
```

2.4.9 Menampilkan Linked List Awal

- Digunakan untuk menampilkan isi linked list sebelum penghapusan dilakukan.

```
// cetak list awal
System.out.println("List awal: ");
printList_3030(head_3030);
```

2.4.10 Menghapus Head

- Digunakan untuk menghapus node pertama pada linked list lalu menampilkan hasilnya.

```
// hapus head
head_3030 = deleteHead_3014(head_3030);
System.out.println("List setelah head dihapus: ");
printList_3030(head_3030);
```

2.4.11 Menghapus Node Terakhir

- Digunakan untuk menghapus node paling akhir pada linked list lalu menampilkan hasilnya.

```
// hapus node terakhir
head_3030 = removeLastNode_3014(head_3030);
System.out.println("List setelah simpul terakhir dihapus: ");
printList_3030(head_3030);
```

2.4.12 Menghapus Node posisi Tertentu

- Digunakan untuk menghapus node pada posisi ke-2 lalu menampilkan hasilnya.

```
// hapus node di posisi 2
int position_3030 = 2;
head_3030 = deleteNode_3014(head_3030, position_3030);
System.out.println("List setelah posisi 2 dihapus: ");
printList_3030(head_3030);
}
```

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan seluruh program yang telah dibuat, dapat disimpulkan bahwa struktur data *Single Linked List* digunakan untuk menyimpan data secara berantai menggunakan node yang saling terhubung.

Program yang dibuat mencakup beberapa operasi dasar pada *Single Linked List*, yaitu:

- pembuatan node,
- penelusuran (*traversal*),
- pencarian data,
- penambahan node di depan, belakang, dan posisi tertentu,
- serta penghapusan node pada bagian tertentu.

Dengan praktikum ini, dapat dipahami cara kerja pointer pada linked list dan bagaimana data dapat ditambahkan, dicari, ditampilkan, maupun dihapus secara dinamis. Selain itu, praktikum ini juga membantu meningkatkan pemahaman logika pemrograman dan implementasi struktur data dalam bahasa Java.

TUGAS PRATIKUM

“Pasien_2511533030”

Kode program :

```
package pekan5_2511533030;
```

```

public class Pasien_2511533030 {
    private String namaPasien_3030;
    private String penyakit_3030;
    private int nomorAntrian_3030;
    Pasien_2511533030 next_3030;

    public Pasien_2511533030(String namaPasien_3030, String penyakit_3030, int nomorAntrian_3030) {
        this.namaPasien_3030 = namaPasien_3030;
        this.penakit_3030 = penyakit_3030;
        this.nomorAntrian_3030 = nomorAntrian_3030;
        this.next_3030 = null;
    }

    public String getNamaPasien_3030() { return namaPasien_3030; }
    public String getPenyakit_3030() { return penyakit_3030; }
    public int getNomorAntrian_3030() { return nomorAntrian_3030; }
    public Pasien_2511533030 getNext_3030() { return next_3030; }

    public void setNext_3030(Pasien_2511533030 next_3030) { this.next_3030 = next_3030; }
}

```

“RumahSakit_2511533030”

```

package pekan5_2511533030;

import java.util.Scanner;

public class RumahSakit_2511533030 {
    private Pasien_2511533030 head_3030;
    private int counter_3030;

    public RumahSakit_2511533030() {
        head_3030 = null;
        counter_3030 = 0;
    }

    public void daftarkanPasien_3030(String nama_3030, String penyakit_3030) {
        counter_3030++;
        Pasien_2511533030 newPasien_3030 = new Pasien_2511533030(nama_3030,
        penyakit_3030, counter_3030);
        if (head_3030 == null) head_3030 = newPasien_3030;
        else {
            Pasien_2511533030 temp_3030 = head_3030;
            while (temp_3030.getNext_3030() != null) temp_3030 =
temp_3030.getNext_3030();
            temp_3030.setNext_3030(newPasien_3030);
        }
        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
        counter_3030);
    }

    public void panggilPasien_3030() {
        if (head_3030 == null) {
            System.out.println("Antrian kosong!");
            return;
        }
    }
}

```

```

    }
    System.out.println("Memanggil Pasien: " + head_3030.getNamaPasien_3030() +
        " | Keluhan: " + head_3030.getPenyakit_3030() +
        " | Nomor Antrian: " + head_3030.getNomorAntrian_3030());
    head_3030 = head_3030.getNext_3030();
}

public void tampilkanAntrian_3030() {
    if (head_3030 == null) { System.out.println("Antrian kosong!"); return; }
    Pasien_2511533030 temp_3030 = head_3030;
    System.out.println("=== Daftar Antrian Pasien ===");
    while (temp_3030 != null) {
        System.out.println("Nomor: " + temp_3030.getNomorAntrian_3030() +
            " | Nama: " + temp_3030.getNamaPasien_3030() +
            " | Keluhan: " + temp_3030.getPenyakit_3030());
        temp_3030 = temp_3030.getNext_3030();
    }
}

public void cariPasien_3030(String nama_3030) {
    Pasien_2511533030 temp_3030 = head_3030;
    while (temp_3030 != null) {
        if (temp_3030.getNamaPasien_3030().equalsIgnoreCase(nama_3030)) {
            System.out.println("Pasien ditemukan! Nomor Antrian: " +
temp_3030.getNomorAntrian_3030() +
                " | Nama: " + temp_3030.getNamaPasien_3030() +
                " | Keluhan: " + temp_3030.getPenyakit_3030());
            return;
        }
        temp_3030 = temp_3030.getNext_3030();
    }
    System.out.println("Pasien dengan nama " + nama_3030 + " tidak
ditemukan.");
}

public void cekStatusAntrian_3030() {
    if (head_3030 == null) { System.out.println("Antrian kosong!"); return; }
    int jumlah_3030 = 0;
    Pasien_2511533030 temp_3030 = head_3030;
    while (temp_3030 != null) { jumlah_3030++; temp_3030 =
temp_3030.getNext_3030(); }
    System.out.println("Jumlah pasien: " + jumlah_3030);
    System.out.println("Pasien terdepan: " + head_3030.getNamaPasien_3030() +
        " | Keluhan: " + head_3030.getPenyakit_3030());
}

public static void main(String[] args) {
    Scanner sc_3030 = new Scanner(System.in);
    RumahSakit_2511533030 rs_3030 = new RumahSakit_2511533030();
    int pilihan_3030;

    do {
        System.out.println("\n=== Antrian Rumah Sakit SLL NIM: 2511533030
===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
    } while (true);
}

```

```

        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan_3030 = sc_3030.nextInt();
        sc_3030.nextLine();

        switch (pilihan_3030) {
            case 1 -> {
                System.out.print("Masukkan Nama Pasien: ");
                String nama_3030 = sc_3030.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String penyakit_3030 = sc_3030.nextLine();
                rs_3030.daftarkanPasien_3030(nama_3030, penyakit_3030);
            }
            case 2 -> rs_3030.panggilPasien_3030();
            case 3 -> rs_3030.tampilkanAntrian_3030();
            case 4 -> {
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                String cari_3030 = sc_3030.nextLine();
                rs_3030.cariPasien_3030(cari_3030);
            }
            case 5 -> rs_3030.cekStatusAntrian_3030();
            case 6 -> System.out.println("Program selesai.");
            default -> System.out.println("Pilihan tidak valid!");
        }
    } while (pilihan_3030 != 6);
    sc_3030.close();
}
}

```

Output program

```

=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan:

```

```

=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Aisyah
Masukkan Keluhan: Sakit perut
Pasien berhasil didaftarkan! Nomor Antrian: 1

```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Dinda
Masukkan Keluhan: Patah hati
Pasien berhasil didaftarkan! Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian Pasien ===
Nomor: 1 | Nama: Aisyah | Keluhan: Sakit perut
Nomor: 2 | Nama: Dinda | Keluhan: Patah hati
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian Pasien ===
Nomor: 2 | Nama: Dinda | Keluhan: Patah hati
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien: Dinda | Keluhan: Patah hati | Nomor Antrian: 2
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Antrian kosong!
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status Antrian  
6. Keluar  
Pilihan: 4  
Masukkan Nama Pasien yang dicari: 6  
Pasien dengan nama 6 tidak ditemukan.
```

```
=== Antrian Rumah Sakit SLL NIM: 2511533030 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status Antrian  
6. Keluar  
Pilihan: 5  
Antrian kosong!
```